

Çalışma Kılavuzu — Proje Nasıl Başlatılır ve Yürütülür

Bariş Köse

August 24, 2026

CONTENTS

Çalışma Kılavuzu	
BÖLÜM 1 — Terimler	
Doküman ve girdi terimleri	
Bu üç belge fiilen neye benziyor	
Teknik terimler	
Migration nedir — açık hâli	
Ortam değişkeni nedir — açık hâli	
Süreç terimleri	
BÖLÜM 2 — Yeni projeye başlamadan önce	
2.1 Hazırlık	
2.2 Cevaplarını önceden düşün	
BÖLÜM 3 — Kurulum: /yeni-proje	
BÖLÜM 4 — Kurulumdan sonra: bir oturum nasıl geçer	
Bir oturumun ritmi	
Neden her adımda /clear	
BÖLÜM 5 — Hangi dosya ne işe yarıyor	
Kök dizin	
docs/standards/ — her projede aynı	
docs/project/ — her projede var, içeriği projeye özel	
BÖLÜM 6 — Hangi komut ne zaman	
Pencere yenileme — nereye tıklanır	

/kit-senkron ne yapar

BÖLÜM 7 — Takıldığında

BÖLÜM 8 — Öğrendiklerim defteri

Kim yazar

Ne nereye gider

Ne işe yarıyor

BÖLÜM 9 — Bu kılavuz nasıl kısaltılır

Çalışma Kılavuzu

Bu dosya kullanıcı içindir. Ajanın uyacağı kurallar `CLAUDE.md` ve `docs/standards/` içinde; burada **projenin nasıl yürütüleceği** anlatılıyor.

★ **Bölüm 1 (terimler) zamanla silinebilir.** Terimler oturduğunda o bölümü kaldır, kılavuz kısalsın. Diğer bölümler kalıcıdır.

BÖLÜM 1 — Terimler

Akış boyunca geçen kelimeler. Bilinmeyen bir terim, verilen cevabı da geçersiz kılar.

Doküman ve girdi terimleri

Terim	Ne demek	Somut örnek
Analiz dokümanı	Projenin ne yapması istendiğini anlatan, sana verilen belge	<code>analiz.docx</code> , şartname, ihale dosyası, talep formu
PRD (Product Requirements Document)	"Sistem tam olarak ne yapacak" sorusunun yazılı ve eksiksiz hâli. Analiz dokümanından üretilir	Roller, iş kuralları, kapsam dışı, hata durumları
API dokümanı	Var olan bir servisin nasıl çağrılacağını anlatan belge	Aşağıda açıldı
Veritabanı şeması	Var olan bir veritabanında hangi tablolar ve ilişkiler olduğunu gösteren belge	Aşağıda açıldı

BU ÜÇ BELGE FİLEN NEYE BENZİYOR

Analiz dokümanı genelde Word veya PDF. İçinde: projenin amacı, kimlerin kullanacağı, hangi ekranların olacağı, hangi kuralların işleyeceği.

⚠ Her zaman eksiktir. "İş emri kapatılabilmelidir" yazar ama "kim kapatabilir, hangi durumdayken, açıklama zorunlu mu" yazmaz. PRD görüşmesi tam da bu boşlukları kapatmak içindir.

API dokümanı, var olan bir servisi kullanacaksan gerekir. Üç şeyi söyler: hangi adrese istek atılacak, ne gönderilecek, ne dönecek.

```
GET /personel/{id}
Dönen: { "ad": "...", "soyad": "...", "birim": "...", "durum": "aktif" }
```

Genelde Swagger/OpenAPI sayfası, Postman koleksiyonu veya bir Word dosyası olarak gelir. **Yoksa** kurumdan istenir; onsuz o servise bağlanılamaz.

Veritabanı şeması, var olan bir veritabanına bağlanacaksan gerekir. Hangi tablolar var, içlerinde hangi kolonlar, tablolar birbirine nasıl bağlı.

```
Tablo: personel
  id          (sayı, birincil anahtar)
  ad          (metin)
  birim_id    (sayı) → birim tablosuna bağlı
```

Kutucuk-ok çizimi (ERD), tablo listesi veya `CREATE TABLE` komutları hâlinde gelebilir.

⚠ **Üçü de yoksa sorun değil** — yeni bir sistem kuruyorsan zaten olmaz. Kit soru sorarak eksiği tamamlar. | **ADR** (Architecture Decision Record) | Önemli bir teknik kararın **neden** alındığını yazan kısa belge | *"Repository Pattern kullanmadık, çünkü..."* | **Roadmap** | Yapılacak adımların **sırayla** listesi, kutucuklu | *"Adım 4 — kimlik doğrulama ✓"* |

⚠ **Analiz dokümanı ile PRD karıştırılmaz.** Analiz **girdidir**, sana verilir ve eksiktir. PRD **çıktıdır**, sorular sorularak üretilir ve eksiği kalmaz.

Teknik terimler

Terim	Ne demek
İstemci / tüketici	API'den veri çeken program: web arayüzü, mobil uygulama, başka kurumun sistemi
Uç nokta (endpoint)	Uygulamanın dışarıya açtığı bir adres: <i>"buraya şunu gönderirsen şunu yaparım"</i>
Migration	Veritabanının yapısını değiştiren komut dosyaları — aşağıda açıldı
Seed	Geliştirme için üretilen sahte örnek veri
Konteyner	Uygulamayı ihtiyaçlarıyla birlikte paketleyip çalıştıran kutu (Docker)
Ortam değişkeni	Koda yazılmayan, dışarıdan verilen ayar — aşağıda açıldı
CI	Kod gönderildiğinde otomatik çalışan test ve derleme zinciri
Deploy	Kodun, kullanıcıların erişebildiği bir yerde çalışır hâle getirilmesi
İzleme (monitoring)	Sistem canlıdayken neyin yavaşladığını, neyin hata verdiğini gösteren araçlar

MIGRATION NEDİR — AÇIK HÂLİ

Veritabanının **yapısı** değiştiğinde (yeni tablo, yeni kolon, silinen kolon) bu değişiklik elle yapılmaz. Bunun yerine bir **dosya** üretilir.

Dosya nerede durur: `prisma/migrations/` klasöründe, tarih damgalı:

```
prisma/migrations/
├─ 20260101120000_ilk_tablolar/migration.sql
├─ 20260115093000_is_emrine_foto_alani_ekle/migration.sql
├─ 20260203141500_lokasyona_index_ekle/migration.sql
```

"Sırayla çalışır" ne demek: Üçüncü dosya, ilk ikisinin çalıştığını varsayar. "Fotoğraf alanı ekle" komutu ancak tablo daha önce oluşturulmuşsa anlamlıdır. Bu yüzden dosya adları tarih damgalı — sıra kaybolmasın diye.

”Kayıt altına alınır” ne demek: İki şey birden:

1. Dosyalar **git’e girer**, yani sen, DevOps ve otomatik testler **aynı** komutları çalıştırır. *”Bende tablo var sende yok”* durumu oluşmaz
2. Veritabanı kendi içinde **hangilerinin çalıştığını** bir tabloda tutar. Komut ikinci kez çalıştırılmaz

Sen ne yapıyorsun: Şemayı değiştirip komutu veriyorsun; dosya kendiliğinden üretiliyor. SQL yazmıyorsun.

⚠ Canlı ortamda yalnızca *”uygula”* komutu çalışır, *”üret”* komutu asla — ikincisi veri kaybettirebilir.

ORTAM DEĞİŞKENİ NEDİR — AÇIK HÂLİ

Aynı kod farklı yerlerde farklı ayarlarla çalışır: senin bilgisayarında başka veritabanı, canlıda başka. Bu ayarlar **koda yazılmaz**, dışarıdan verilir.

Nereye yazılır: Proje kökündeki `.env` dosyasına, `AD=değer` biçiminde:

```
DATABASE_URL=postgres://kullanici:sifre@localhost:55432/bakim
JWT_SECRET=uzun-rastgele-bir-metin
WEB_PORT=3100
```

Kim okur: Uygulama açılırken bu dosyayı okur ve değerleri alır.

Neden koda yazılmıyor — iki sebep:

1. **Değer ortama göre değişir.** Senin makinende `localhost`, canlıda kurumun sunucusu. Kod aynı kalmalı, yalnızca değer değişmeli
2. 🚫 **İçinde şifre var.** Koda yazılırsa git’e girer; git geçmişi silinmez, yani bir kez girdiyse **sonsuz kadar orada kalır**

Süreç terimleri

Terim	Ne demek
Dal (branch)	Ana koda dokunmadan çalışılan kopya
Birleştirme isteği (PR / MR)	<i>"Bu dalı ana koda alalım mı"</i> önerisi; inceleme burada yapılır
Oturum (session)	Ajanla yapılan bir çalışma turu. Bir roadmap adımı = bir oturum
Devir notu	Oturum kapanırken yazılan <i>"sırada ne var"</i> notu

BÖLÜM 2 — Yeni projeye başlamadan önce

2.1 Hazırlık

1. **Boş bir klasör** aç ve VS Code'da aç
2. **Kiti güncelle** — her yeni projede, atlanmaz. Sohbete yapıştır:

```
claude plugin update proje-kiti@bariskose-skills
```

Eklentinin tam adı `proje-kiti@bariskose-skills` — `proje-kiti` eklenti adı, `bariskose-skills` ise geldiği kaynak.

Sonra pencereyi yenile (aşağıda anlatıldı).

⚠ Kite sürekli yeni kural ekleniyor. Eski sürümle başlarsan o kurallar projeye hiç gelmez.

3. **Elindeki belgeleri klasöre koy** — varsa:

Belge	Ne zaman gerekir	Yoksa
Analiz dokümanı	Her zaman — projenin ne yapacağı burada	Sözlü anlatırsın, kit yazıya döker
API dokümanı	Var olan bir servise bağlanacaksan	Yeni sistemde gerekmez
Veritabanı şeması	Var olan bir veritabanına bağlanacaksan	Yeni sistemde gerekmez

⚠ Bu belgeler **girdi**dir; sen yazmazsın, sana verilir. İçeriklerinin neye benzediği Bölüm 1'de açıklandı.

2.2 Cevaplarını önceden düşün

Kurulumda şunlar sorulacak. Şimdiden düşünmek görüşmeyi hızlandırır:

Soru	Ne cevaplayacaksın
Bu proje kimin için?	İşyeri projesi mi, kendi projen mi
Proje tipi?	Web · mobil · ikisi
API'yi senin yazmadığın biri tüketecek mi?	Başka kurum, yüklenici, merkezî sistem
Kendiliğinden çalışması gereken iş var mı?	Zamanlanmış görev, gece raporu
Kod nerede duracak, deploy'u kim yapacak?	GitHub/GitLab · sen/DevOps

BÖLÜM 3 — Kurulum: `/yeni-proje`

Tek komut:

```
/yeni-proje
```

Gerisi sohbet. Sırayla şunlar olur:

Adım	Ne oluyor	Senden ne isteniyor
0	Eklentiler kontrol edilir, platform tespit edilir, ses bildirimi kurulur	İzin
1	Kim için · proje tipi · backend kurgusu (4 soru) · API biçimi (4 soru) · proje adı	Cevaplar
2	Kit dosyaları projeye kopyalanır	—
3	★ PRD görüşmesi — en uzun adım	Analiz dokümanını verirsin, tek tek soru cevaplırsın
4	Yol haritası, ilk kararlar, teknoloji-ve-plan iskeleti	Onay
5	İskelet kurulur — ilk kod	Onay
6	Yayın veya teslim paketi (1. adımdaki cevaba göre)	Duruma göre
7	Son kontrol	—

⚠ **Bu tek komutluk bir işlem değil.** `/yeni-proje` kurulumu başlatır; Adım 3 uzun bir görüşmedir. Vaat *"tek promptla uygulama"* değil, **"doğru kurulmuş proje ve net yol haritası"**.

🛑 **Adım 3'te acele etme.** Analiz dokümanında yazmayan onlarca karar orada netleşir. Cevabı bilinmeyen bir kural kodlanırsa yanlış varsayım tüm katmanlara yayılır.

BÖLÜM 4 — Kurulumdan sonra: bir oturum nasıl geçer

Kurulum bitince artık kit değil, projedeki `CLAUDE.md` ve `docs/standards/` geçerlidir. İlerleme **roadmap adımlarıyla** olur.

Bir oturumun ritmi

- | | |
|-------------------------|---|
| 1. Aç ve devral edelim" | → "docs/project/sonraki-adim-prompt.md oku, devam |
| 2. Plan sun | → ajan ne yapacağını anlatır |
| 3. Onayla | → gerekiyorsa düzelt |
| 4. Kod + test | → ajan yazar, testler yeşil olur |
| 5. Gözle doğrula | → ekran varsa tarayıcıda görülür |
| 6. Commit + öneri | → değişiklik önerisi açılır |
| 7. Kutucuk ✓ | → roadmap'te o adım işaretlenir |
| 8. Kararları yaz | → teknoloji-ve-plan.md güncellenir |
| 9. Devir notu | → sırada ne var yazılır |
| 10. /clear | → yeni oturuma temiz başla |

🚫 **7 ve 8 atlanmaz.** Kutucuk *nerede kalındığını*, teknoloji belgesi *neden öyle yapıldığını* söyler. İkisi de sonradan hatırlanmaz.

Neden her adımda `/clear`

Konuşma uzadıkça ajanın bağlamı dolar ve ayrıntı kaybolur. Her adım kendi oturumunda yapılır; devir notu sayesinde hiçbir şey kaybolmaz.

BÖLÜM 5 — Hangi dosya ne işe yarıyor

`/yeni-proje` bittiğinde projede şunlar olur:

Kök dizin

Dosya	Ne işe yarıyor	Kim okur
<code>CLAUDE.md</code>	Ajanın uyacağı çalışma protokolü	Ajan
<code>CALISMA-KILAVUZU.md</code>	Bu dosya — projenin nasıl yürütüleceği	Sen
<code>REPO-YAPISI.md</code>	Hangi klasörde ne var, hangi iş nerede yapılıyor	İkisi
<code>README.md</code>	Projeyi kuran/çalıştıran için: kurulum, komutlar, ortam değişkenleri	Devralan geliştirici, DevOps
<code>.env.example</code>	Hangi ayarların gerektiğinin listesi — değerler boş	DevOps, devralan geliştirici
<code>.env</code>	O ayarların gerçek değerleri — 🚫 asla commit edilmez	Sadece o makine

Bu ikisi neden ayrı — somut örnek

`.env.example` git'e girer, herkes görür. Ayarların **adlarını** söyler, değerlerini değil:

```
DATABASE_URL=
JWT_SECRET=
WEB_PORT=
```

`.env` git'e **girmez**, yalnızca o makinede durur. Gerçek değerler burada:

```
DATABASE_URL=postgresql://kullanici:GercekSifre123@localhost:55432/bakim
JWT_SECRET=a8f3c91e7b2d4056
WEB_PORT=3100
```

★ **Faydası:** Projeyi devralan biri `.env.example`'a bakıp *"demek ki üç ayar gerekiyormuş"* der ve kendi değerlerini yazar. Tahmin etmesi gerekmez.

🚫 `.env` neden commit edilmez: içinde şifre var ve **git geçmişi silinmez**. Bir kez commit edildiyse sonradan dosyayı silsen bile geçmişte durur; o depoya erişen herkes o şifreyi görebilir. Tek çözüm şifreyi değiştirmektir.

`docs/standards/` — her projede aynı

Mühendislik kuralları: nasıl kod yazılır, hangi teknoloji kullanılır, testler nasıl olur. **Bu klasörün içeriği tüm projelerde birebir aynıdır**, çünkü kitten kopyalanıyor.

"Elle değiştirilmez" ne demek: Bir kuralı bu klasörde değiştirirsen yalnızca **bu projede** değişir. Bir sonraki projede eski hâliyle geri gelir — çünkü kaynak kit, bu klasör onun kopyası.

Doğru yol: Kuralı değiştirmek istiyorsan `/kit-senkron` çalıştırırsın; kural kite yazılır ve **bundan sonraki her projeye** kendiliğinden gelir.

Bir kurumun genel yönetmeliğini kendi masandaki kopyanın üstüne yazarak değiştiremezsin — merkeze bildirirsin, oradan herkese dağıtılır.

Dosya	Konu
<code>00-stack.md</code>	Hangi teknoloji kullanılıyor, hangisi kullanılmıyor, neden
<code>01-architecture.md</code>	Katmanlar, klasör yapısı, bağımlılık yönü
<code>02-coding-standards.md</code>	Kod ve yorum yazım kuralları
<code>03-api-guidelines.md</code>	API sözleşmesi, sürümleme, sayfalama
<code>04-database.md</code>	Veri modeli kuralları
<code>05-auth-security.md</code>	Kimlik doğrulama ve güvenlik
<code>06-testing.md</code>	Test stratejisi
<code>08-git-workflow.md</code>	Dal, commit, birleştirme kuralları
<code>09-ci-cd-deploy.md</code>	Otomatik kontroller ve yayın
<code>11-agent-workflow.md</code>	Ajanın nasıl çalışacağı
<code>12-operations-and-scaling.md</code>	Loglama, izleme, ölçekleme
<code>13-environments.md</code>	Local / test / canlı ayrımı
<code>14-privacy-and-compliance.md</code>	KVKK ve kişisel veri
<code>15-oturum-devri.md</code>	Oturum kapanış protokolü
(diğerleri)	Tanım listesi <code>docs/standards/</code> içinde

`docs/project/` — her projede var, içeriği projeye özel

⚠️ *"Bu projeye özel"* demek **"yalnızca bu projede bulunur"** demek değil. Bu klasör **her projede** açılır; içindeki dosya adları da aynıdır. Değişen, **içlerinin dolduruluş biçimidir**.

	<code>docs/standards/</code>	<code>docs/project/</code>
Dosya adları	Her projede aynı	Her projede aynı
İçerik	Her projede aynı	Her projede farklı
Kaynağı	Kitten kopyalanır	Görüşmeyle üretilir
Değişirse	<code>/kit-senkron</code> ile kite yazılır	Doğrudan düzenlenir

Yani otomatiğe bağlanan şey **yapının kendisi**: her projede aynı dokuz dosya açılır, aynı sorular sorulur, aynı sırayla doldurulur. İçlerine yazılan bilgi projeden projeye değişir.

Dosya	Hangi soruya cevap verir	Ne zaman dolar
<code>PRD.md</code>	Sistem ne yapacak	Kurulum Adım 3
<code>roadmap.md</code>	Ne yapılacak, hangi sırayla — kutucuklu	Adım 4, her adımda işaretlenir
<code>teknoloji-ve-plan.md</code>	Neden öyle yapıldı, teknoloji nedir	Adım 4'te açılır, her adımda büyür
<code>decisions/</code>	Önemli kararların gerekçesi (ADR)	Karar alındıkça
<code>data-model.md</code>	Kendi veri modelin	Veri modeli adımımda
<code>integrations.md</code>	Dış sistemlerle nasıl konuşuluyor	Entegrasyon eklendikçe
<code>altyapi-durumu.md</code>	Kod dışında ne yapıldı: hangi hesap açıldı, hangi panelde ne seçildi	Dış işlem yapıldıkça
<code>sonraki-adim-prompt.md</code>	Sırada ne var — yeni oturuma verilir	Her oturum sonunda
<code>ogrendiklerim.md</code>	Senin kişisel defterin : sormayı unuttuğun sorular, zor gelen konular	Ajan sorar, sen onaylarsın
<code>CHANGELOG.md</code>	Sürüm geçmişi	Yayın yapıldıkça

★ **Dört dosya dört ayrı soruya cevap verir, karıştırılmaz:**

Dosya	Cevapladığı soru
<code>PRD.md</code>	Sistem ne yapacak
<code>roadmap.md</code>	Hangi sırayla yapılacak, nerede kalındı
<code>teknoloji-ve-plan.md</code>	Neden öyle yapıldı, teknoloji nedir
<code>altyapi-durumu.md</code>	Kod dışında ne yapıldı

altyapi-durumu.md **açık hâli:** Bir projede kodun dışında da işler yapılır — hesap açılır, panelden ayar yapılır, alan adı satın alınır, veritabanı oluşturulur. Bunlar kodda görünmez ve **kimse hatırlamaz**.

Örnek satırlar:

```
2026-08-24 · Neon'da "bakim-prod" veritabanı açıldı, bölge: Frankfurt
2026-08-24 · Vercel projesine DATABASE_URL değişkeni eklendi (Production)
2026-08-25 · Cloudflare'de bakim.izmir.bel.tr kaydı sunucu IP'sine
yönlendirildi
```

 **Anahtar ve şifre DEĞERLERİ buraya yazılmaz** — yalnızca *"şu değişken şu ortama eklendi"* bilgisi yazılır. Değerler `.env` dosyasında durur.

 Bu dosya olmadan altı ay sonra *"bu ayarı nereden yapmıştım"* sorusunun cevabı kaybolur. Panelde yapılan bir işlemin git geçmişi yoktur.

BÖLÜM 6 — Hangi komut ne zaman

Komut	Ne zaman	Ne yapar
<code>/yeni-proje</code>	Yalnızca boş klasörde , projeye ilk başlarken	Kurulumu baştan sona yürütür
<code>/kit-senkron</code>	Bir kuralı kalıcı hâle getirirken	Aşağıda açıldı
<code>/clear</code>	Her roadmap adımı bitince	Sohbet geçmişini temizler, bağlam sıfırlanır
<code>claude plugin update proje-kiti@bariskose-skills</code>	Her yeni projeden önce	Kitin son sürümünü indirir
Pencere yenileme	Eklenti güncellendikten sonra	Aşağıda açıldı

PENCERE YENILEME — NEREYE TIKLANIR

Eklenti güncellendiğinde **çalışan sürüm hâlâ eskisidir**; pencere yenilenene kadar yeni kurallar devreye girmez.

Fareyle:

- Üst menü çubuğunda **View** (Görünüm)
- Açılan listede **Command Palette...** (Komut Paleti)
- Açılan kutuya yaz: `Reload Window`
- Listede çıkan **Developer: Reload Window** satırına tıkla

Klavyeyle: `Cmd+Shift+P` (Mac) veya `Ctrl+Shift+P` (Windows) → aynı kutu açılır.

En kaba yöntem: VS Code'u tamamen kapatıp yeniden aç. Aynı işi görür.

`/KIT-SENKRON` NE YAPAR

"Kite yaz" demenin **yapılandırılmış** hâli.

Elle söylersen ajan bir dosyaya bir şey ekler ve iş orada biter. `/kit-senkron` ise şunları sırayla yapar:

1. Projenin `docs/standards/` klasörüyle kitteki asıl kopyayı **karşılaştırır**
2. Farkları listeler: *"projede şu kural var, kitte yok"*
3. **Sana sorar:** hangileri kalıcı kural olacak, hangileri bu projeye özel
4. Seçilenleri kite yazar, sürümü yükseltir ve yayınlar
5. Kitteki iyileştirmeleri de **projeye getirir** — iki yön birden

★ Farkı şu: elle yazınca yalnızca bir dosya değişir. `/kit-senkron` ile kural **bir sonraki projede zaten orada olur**.

BÖLÜM 7 — Takıldığında

Belirti	Muhtemel sebep	Ne yapılır
Ajan eski kuralla çalışıyor	Eklenti güncellendi ama pencere yenilenmedi	Reload Window
<code>/yeni-proje</code> beklenmedik davranıyor	Klasör boş değil	Fazla dosyaları taşı veya sil
Ajan cevabını bilmediğim şey soruyor	Terim açıklanmamış	<i>"Bu terimi açıkla"</i> de — kitin kuralı bunu zorunlu tutuyor
Ajan kararı bana bırakıyor	Mühendislik seçimini devretmiş	<i>"Sen karar ver, gerekçesini söyle"</i> de
Nerede kaldığımı hatırlamıyorum	Kutucuk işaretlenmemiş	<code>roadmap.md</code> ve <code>sonraki-adim-prompt.md</code> 'ye bak
Cevaplar yüzeyselleşti	Bağlam dolmuş	<code>/clear</code> yap, devir notuyla devam et

BÖLÜM 8 — Öğrendiklerim defteri

`docs/project/ogrendiklerim.md` senin kişisel defterin. Kite yazılacak kadar genel olmayan ama unutulursa bedeli tekrar ödenecek notlar buraya girer.

Kim yazar

Ajan sorar, sen onaylarsın. Oturum kapanırken şuna benzer bir soru gelir:

"Bu oturumda bağımlılığın tersine çevrilmesi konusunu üçüncü örnekte oturttuğunu fark ettim. Deftere yazayım mı?"

🚫 Senin *"bunu deftere yaz"* demen beklenmiyor. O sırada zaten öğrenmekle meşgulsün; not almayı hatırlaman beklenemez. **Fark eden taraf teklif eder.**

Ne nereye gider

Gözlem	Nereye
<i>"Bu terimi ilk kez anladım"</i>	Defter → zor gelen kararlar
<i>"Şunu sormayı unutmuşuz"</i>	Defter → sormayı unuttuğum sorular
Aynı hata ikinci kez	Defter → tekrar eden hatalar
Aynı hata üçüncü kez	Artık kişisel değil → kite taşınır
<i>"Her projede böyle yapılmalı"</i>	Doğrudan kite

★ **Ayrım basit:** Kite **kural** gider — *"şu durumda şu yapılır."* Deftere **deneyim** girer — *"ben şunu atlamıştım."* Bir deneyim üç kez tekrarlanırsa kurala dönüşür ve kite taşınır.

Ne işe yarıyor

İki şey:

1. **Bir sonraki projenin kontrol listesi.** *"Geçen sefer eş zamanlı kullanıcı sayısını sormayı unutmuşum"* notu, bu sefer sorulmasını sağlar
 2. **İlerlemenin ölçüsü.** *"Zor gelen kararlar"* listesi zamanla kısalır. Kısalması öğrendiğinin kanıtıdır
-

BÖLÜM 9 — Bu kılavuz nasıl kısaltılır

Amaç bu dosyaya bağımlı kalmak değil.

- **Terimler oturduğunda** → Bölüm 1'i sil
- **Akış ezberlendiğinde** → Bölüm 3'ü kısalt, yalnızca komut kalsın
- **Dosya haritası aklında kaldığında** → Bölüm 5'i sil

Kalması gereken tek bölüm: **Bölüm 4 — oturum ritmi**. O, ezberlenmesi değil her seferinde uygulanması gereken bir kontrol listesidir.